Assignment 3

Section A

4. What is GROUP BY and HAVING clause?

GROUP BY Clause

- The GROUP BY clause is used to collect data across multiple records and group the results by one or more columns.
- It is typically used in conjunction with aggregate functions (like SUM, AVG, COUNT) to provide summary information.

HAVING Clause

- The HAVING clause is used to filter the results of a GROUP BY based on a specific condition.
- It acts like a WHERE clause, but specifically for groups rather than individual rows.

5. What is subquery?

Subquery Definition

- A Subquery, or inner query, is a query nested inside another SQL query (the outer query).
- It is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.

Usage and Rules

- **Nesting:** A subquery is always enclosed within parentheses.
- **Placement:** It can be placed in the SELECT, FROM, WHERE, or HAVING clauses.
- **Operators:** It is frequently used with comparison operators such as =, >, <, and logical operators like IN, ANY, or ALL.

Subquery Types

- **Scalar Subquery:** Returns a single value (one row and one column).
- **Multi-row Subquery:** Returns a set of rows (usually used with the IN operator).
- **Correlated Subquery:** A subquery that uses values from the outer query, executing once for each row processed by the main query.

Section B

2. Write a query using JOIN to combine Student and Course tables.

Query Explanation

- To combine the Student and Course tables, we must use the Enrollment table as a bridge because they have a many-to-many relationship.
- An INNER JOIN is used to fetch only the records where there is a match in all three tables.

SQL

```
SELECT
    Student.Name,
    Course.Course_Name
FROM Student
JOIN Enrollment ON Student.Student_ID = Enrollment.Student_ID
JOIN Course ON Enrollment.Course_ID = Course.Course_ID;
```

Join Logic

- **First Join:** Links the Student table to the Enrollment table using the Student_ID.
- **Second Join:** Links the Enrollment table to the Course table using the Course_ID.
- **Result:** This produces a list showing which students are enrolled in which specific courses.

Section C

2. An e-commerce system stores order. Retrieve total sales per customer.

Query Logic

- To calculate total sales, we use the SUM() aggregate function on the order amount.
- The GROUP BY clause is used to group these sums for each individual customer.

- This query assumes an **Orders** table containing Customer_ID and Order_Amount.

SQL

```
SELECT
    Customer_ID,
    SUM(Order_Amount) AS Total_Sales
FROM Orders
GROUP BY Customer_ID;
```

Explanation

- **SUM(Order_Amount):** Adds up all purchase values for each group.
- **GROUP BY Customer_ID:** Ensures the calculation is performed separately for every unique customer.
- **AS Total_Sales:** Renames the resulting column for better readability in the output.